

Congratulations! You can perform simulated annealing. You can see that a random itinerary has distance 16.81, while a nearest neighbor itinerary has distance 6.29, just like we observed before. The perturb search itinerary has distance 7.38, and the simulated annealing itinerary has distance 5.92. In this case, we've found that perturb search performs better than a random itinerary, that nearest neighbor performs better than perturb search and a random itinerary, and simulated annealing performs better than all the others. When you try other random seeds, you may see different results, including cases where simulated annealing does not perform as well as nearest neighbor. This is because simulated annealing is a sensitive process, and several aspects of it need to be tuned precisely in order for it to work well and reliably. After we do that tuning, it will consistently give us significantly better performance than simpler, greedy optimization algorithms. The rest of the chapter is concerned with the finer details of simulated annealing, including how to tune it to get the best possible performance.

METAPHOR-BASED METAHEURISTICS

The peculiarities of simulated annealing are easier to understand if you know its origin. Annealing is a process from metallurgy, in which metals are heated up and then gradually cooled. When the metal is hot, many of the bonds between particles in the metal are broken. As the metal cools, new bonds are formed between particles that lead to the metal having different, more desirable properties. Simulated annealing is like annealing in the sense that when our temperature is hot, we “break” things by accepting worse solutions, in the hope that then, as the temperature cools, we can fix them in a way that makes them better than they were before.

The metaphor is a little contrived, and isn't one that non-metallurgists find intuitive. Simulated annealing is something called a *metaphor-based metaheuristic*. There are many other metaphor-based metaheuristics that take an existing process found in nature or human society and find a way to adapt it to solve an optimization problem. They have names like ant colony optimization, cuckoo search, cuttlefish optimization, cat swarm optimization, shuffled frog leaping, emperor penguins colony, harmony search (based on the improvisation of jazz musicians), and the rain water algorithm. Some of these analogies are contrived and not very useful, but sometimes they can provide or inspire a real insight into a serious problem. In either case, they're nearly always interesting to learn and fun to code.